Python Programming Fundamentals

Inheritance Basics

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1.1 Reusing and extending classes

Inheritance lets a **child class** (subclass) receive all the attributes and methods of a **parent class** (superclass), then add or change them.

Think of it as: "A PremiumMember **is-a** Person."

```
Person
├── FreeUser
└── PremiumMember
```

Benefits:

- Avoid repeating code in every subclass
- Model real-world hierarchies naturally
- Supports polymorphism

Outline

2. Python Syntax

```
class Person:                          # parent / superclass
    def __init__(self, name: str, email: str):
        self.__name = name
        self.__email = email

    def get_name(self):  return self.__name
    def get_email(self): return self.__email

    def __str__(self) -> str:
        return f"Person: {self.__name} <{self.__email}>"


class FreeUser(Person):                # child / subclass
    pass                               # inherits everything from Person
```

`class FreeUser(Person):` — the `(Person)` part declares the parent.

Outline

3. Inherited Attributes and Methods

```
class FreeUser(Person):
    pass

# FreeUser inherits __init__, get_name, get_email, __str__
user = FreeUser("Alice", "alice@example.com")
print(user.get_name())    # Alice
print(user.get_email())   # alice@example.com
print(user)               # Person: Alice <alice@example.com>
```

Even though `FreeUser` has no code of its own, it behaves like a `Person`.

Outline

4. Adding Attributes in a Subclass

```
class PremiumMember(Person):
    def __init__(self, name: str, email: str,
                 subscription: str):
        super().__init__(name, email)     # call parent __init__
        self.__subscription = subscription

    def get_subscription(self) -> str:
        return self.__subscription

    def __str__(self) -> str:
        return (f"PremiumMember: {self.get_name()} "
                f"[{self.__subscription}]")
```

`super().__init__(name, email)` calls the **parent** constructor to set up the inherited attributes.

Outline

5. The Social Network Example

```
Person
  ├── name
  ├── email
  ├── get_name() / get_email()
  └── __str__()
        │
        ├── FreeUser(Person)
        │      └── max_posts: int
        │
        └── PremiumMember(Person)
               └── subscription: str (Gold/Platinum)
```

Outline

6. Overriding Methods

```
class FreeUser(Person):
    def __init__(self, name: str, email: str,
                 max_posts: int = 10):
        super().__init__(name, email)
        self.__max_posts = max_posts

    def __str__(self) -> str:
        return (f"FreeUser: {self.get_name()} "
                f"(max {self.__max_posts} posts/month)")

p = FreeUser("Bob", "bob@example.com", 5)
print(p)
# FreeUser: Bob (max 5 posts/month)
```

The child's `__str__` **overrides** the parent's — the child version is called instead.

Outline

7. Using the Classes

```
from person import Person
from free_user import FreeUser
from premium_member import PremiumMember

users = [
    FreeUser("Alice", "alice@example.com", 10),
    PremiumMember("Bob", "bob@example.com", "Gold"),
    FreeUser("Carol", "carol@example.com", 5),
]

for user in users:
    print(user)         # calls each class's own __str__
    print(user.get_email())   # inherited from Person
```

Thanks for Watching - Any questions?